Лабораторная работа №10

Программирование в командном процессоре ОС UNIX

Студент: Аньсяоянь
Группа: НПИбд-01-25
Номер студенческого билета: 1132258191
Дата выполнения: 2026.04.10

---

1. Цель работы

Освоить основы программирования в командной оболочке `bash` ОС UNIX/Linux.
Научиться создавать командные файлы (скрипты), использовать переменные, арифметические операции, управляющие конструкции (`for`, `if`, `while`), обрабатывать параметры командной строки, работать с файлами и каталогами.

---

2. Выполнение заданий

2.1. Задание 1. Резервное копирование самого себя

Условие:
Создать скрипт, который при запуске делает резервную копию самого себя в директорию `~/backup` с использованием одного из архиваторов (`tar`, `zip` или `bzip2`).

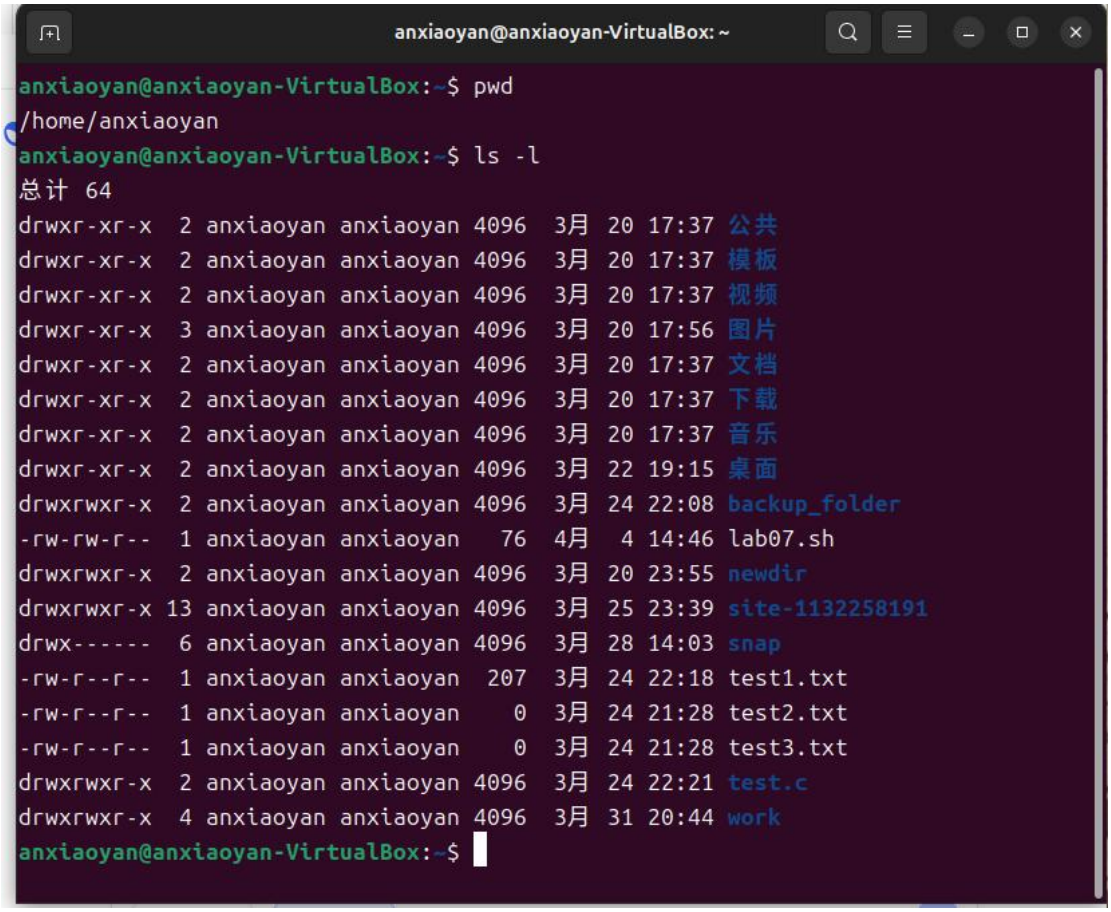
Листинг скрипта `backup_self.sh`:

bash

```
#!/bin/bashSCRIPT_PATH="$0"BACKUP_DIR="$HOME/backup"mkdir -p "$BACKUP_DIR"tar -czf "$BACKUP_DIR/backup_self.tar.gz" "$SCRIPT_PATH"echo "Backup saved to $BACKUP_DIR/backup_self.tar.gz"
```

Результат выполнения:

https://screenshot1.png



На скриншоте видно:

-

Запуск скрипта `./backup_self.sh`

-
-

Сообщение об успешном создании архива

-
-

Проверка содержимого `~/backup` через `ls -l ~/backup`

-
-

Просмотр содержимого архива через `tar -tzf`

-

2.2. Задание 2. Обработка произвольного числа аргументов (более 10)

Условие:
Написать скрипт, который обрабатывает любое количество аргументов командной строки (в том числе более 10) и выводит их значения.

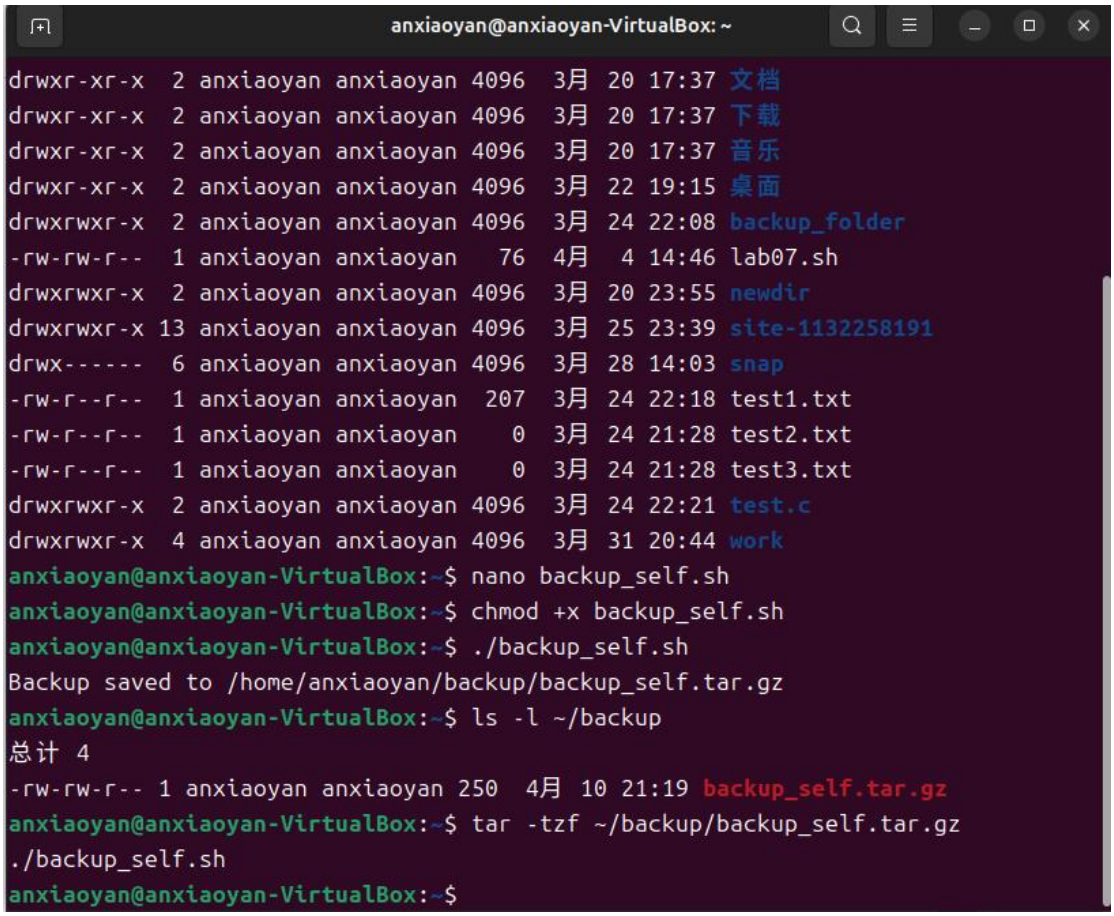
Листинг скрипта `print_args.sh`:

bash

```
#!/bin/bashecho "Total arguments: $#"for arg in "$@"do    echo "$arg"done
```

Результат выполнения:

https://screenshot2.png



На скриншоте видно:

-

Вызов `./print_args.sh a b c d e f g h i j k l m n o`

-
-

Вывод `Total arguments: 15`

-
-

Последовательный вывод всех 15 аргументов

-

2.3. Задание 3. Аналог команды `ls` (без использования `ls` и `dir`)

Условие:
Создать скрипт, который выводит содержимое указанного каталога (или текущего, если каталог не задан) и отображает права доступа к каждому файлу.

Листинг скрипта `myls.sh`:

bash

```
#!/bin/bashTARGET_DIR="${1:-.}"for file in "$TARGET_DIR"/*do    if [ -e "$file" ]; then        perms="
"        [ -r "$file" ] && perms="${perms}r" || perms="${perms}-"        [ -w "$file" ] && perms="${per
ms}w" || perms="${perms}-"        [ -x "$file" ] && perms="${perms}x" || perms="${perms}-"        echo
"$perms $file"    fidone
```

Результат выполнения:

https://screenshot3.png

```
anxiaoyan@anxiaoyan-VirtualBox: ~
总计 4
-rw-rw-r-- 1 anxiaoyan anxiaoyan 250  4月 10 21:19 backup_self.tar.gz
anxiaoyan@anxiaoyan-VirtualBox:~$ tar -tzf ~/backup/backup_self.tar.gz
./backup_self.sh
anxiaoyan@anxiaoyan-VirtualBox:~$ nano print_args.sh
anxiaoyan@anxiaoyan-VirtualBox:~$ chmod +x print_args.sh
anxiaoyan@anxiaoyan-VirtualBox:~$ ./print_args.sh a b c d e f g h i j k l m n o
Total arguments: 15
a
b
c
d
e
f
g
h
i
j
k
l
m
n
o
anxiaoyan@anxiaoyan-VirtualBox:~$
```

https://screenshot4.png

```
anxiaoyan@anxiaoyan-VirtualBox: ~
anxiaoyan@anxiaoyan-VirtualBox:~$ chmod +x myls.sh
anxiaoyan@anxiaoyan-VirtualBox:~$ ./myls.sh
rwx ./公共
rwx ./模板
rwx ./视频
rwx ./图片
rwx ./文档
rwx ./下载
rwx ./音乐
rwx ./桌面
rwx ./backup
rwx ./backup_folder
rwx ./backup_self.sh
rw- ./lab07.sh
rwx ./myls.sh
rwx ./newdir
rwx ./print_args.sh
rwx ./site-1132258191
rwx ./snap
rw- ./test1.txt
rw- ./test2.txt
rw- ./test3.txt
rwx ./test.c
rwx ./work
```

На скриншотах видно:

-

Вывод содержимого текущего каталога с правами доступа (например, `rwx ./myls.sh`, `rw- ./backup_self.sh`)

-
-

Вывод содержимого каталога `/tmp` с правами доступа

-

2.4. Задание 4. Подсчёт файлов по расширению

Условие:

Скрипт получает два аргумента: расширение файла (например `.txt`) и путь к каталогу. Выводит количество файлов с данным расширением в указанном каталоге.

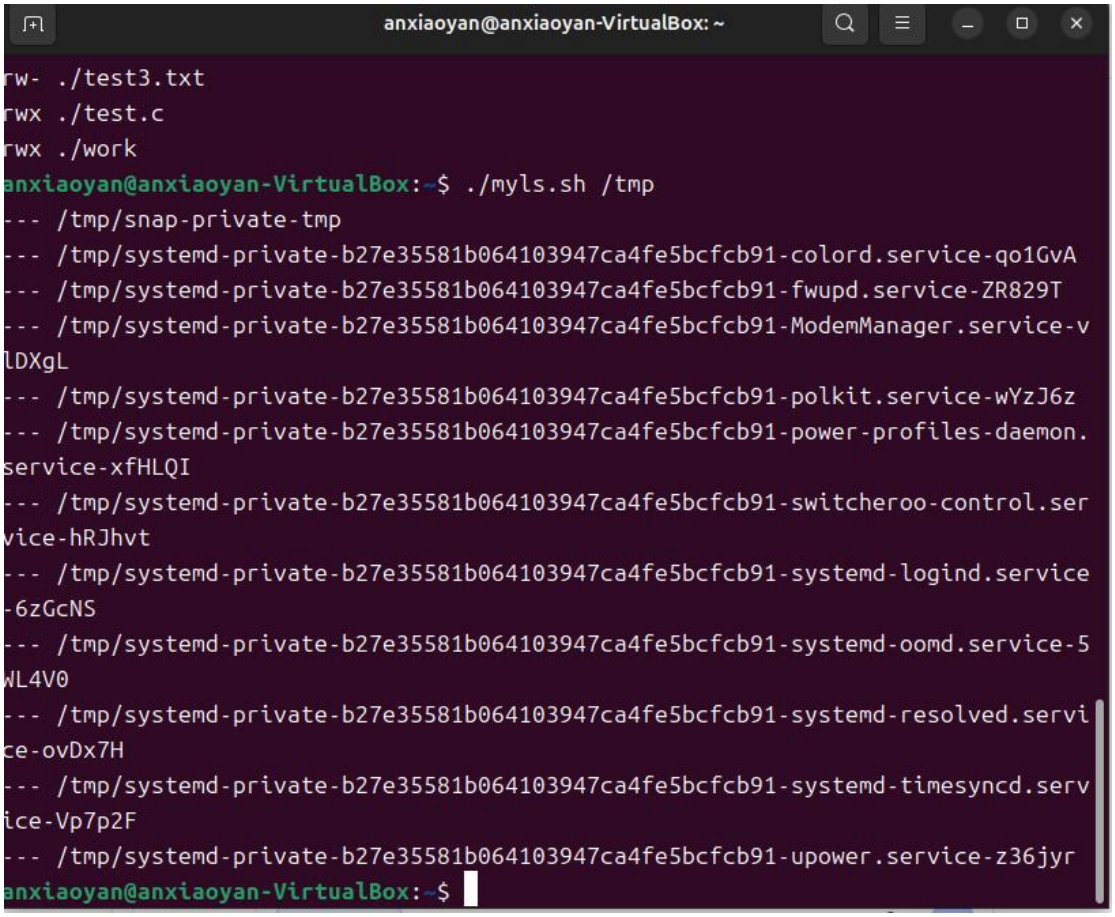
Листинг скрипта `count_files.sh`**:**

bash

```
#!/bin/bashEXT="$1"DIR="$2"if [ -z "$EXT" ] || [ -z "$DIR" ]; then    echo "Usage: $0 <extension> <dir
ectory>"    exit 1fiif [ ! -d "$DIR" ]; then    echo "Error: $DIR is not a directory"    exit 1ficount=
$(find "$DIR" -type f -name "*$EXT" | wc -l)echo "Number of *$EXT files in $DIR: $count"
```

Результат выполнения:

https://screenshot5.png



https://screenshot6.png

```
anxiaoyan@anxiaoyan-VirtualBox: ~
--- /tmp/snap-private-tmp
--- /tmp/systemd-private-b27e35581b064103947ca4fe5bcfcb91-colord.service-qo1GvA
--- /tmp/systemd-private-b27e35581b064103947ca4fe5bcfcb91-fwupd.service-ZR829T
--- /tmp/systemd-private-b27e35581b064103947ca4fe5bcfcb91-ModemManager.service-v
lDXgL
--- /tmp/systemd-private-b27e35581b064103947ca4fe5bcfcb91-polkit.service-wYzJ6z
--- /tmp/systemd-private-b27e35581b064103947ca4fe5bcfcb91-power-profiles-daemon.
service-xfHLQI
--- /tmp/systemd-private-b27e35581b064103947ca4fe5bcfcb91-switcheroo-control.ser
vice-hRJhvt
--- /tmp/systemd-private-b27e35581b064103947ca4fe5bcfcb91-systemd-logind.service
-6zGcNS
--- /tmp/systemd-private-b27e35581b064103947ca4fe5bcfcb91-systemd-oomd.service-5
WL4V0
--- /tmp/systemd-private-b27e35581b064103947ca4fe5bcfcb91-systemd-resolved.servi
ce-ovDx7H
--- /tmp/systemd-private-b27e35581b064103947ca4fe5bcfcb91-systemd-timesyncd.serv
ice-Vp7p2F
--- /tmp/systemd-private-b27e35581b064103947ca4fe5bcfcb91-upower.service-z36jyr
anxiaoyan@anxiaoyan-VirtualBox:~$ nano count_files.sh
anxiaoyan@anxiaoyan-VirtualBox:~$ chmod +x count_files.sh
anxiaoyan@anxiaoyan-VirtualBox:~$ ./count_files.sh .txt ~
Number of *.txt files in /home/anxiaoyan: 16
anxiaoyan@anxiaoyan-VirtualBox:~$
```

На скриншотах видно:

- `./count_files.sh .txt ~` → вывод количества `.txt` файлов
-
- `./count_files.sh .sh .` → вывод количества `.sh` файлов в текущем каталоге
-

---

3. Контрольные вопросы и ответы на них

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются?

Ответ:
Командная оболочка (shell) — это программа, которая принимает команды пользователя, интерпретирует их и передаёт операционной системе для выполнения.
Примеры: `sh` (Bourne shell), `bash` (Bourne Again shell), `csh` (C shell), `ksh` (Korn shell), `zsh`.
Отличия: синтаксис, наличие истории команд, поддержка массивов, автодополнение, встроенные арифметические функции, настройка приглашения.

2. Что такое POSIX?

Ответ:
POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов IEEE, описывающих интерфейсы между ОС и прикладными программами.
Обеспечивает переносимость программ на уровне исходного кода между UNIX-подобными системами.

3. Как определяются переменные и массивы в языке программирования bash?

Ответ:
Переменные: `имя=значение` (без пробелов). Использование: `$имя` или `${имя}`.
Массивы: `set -A имя значение1 значение2 ...` или `имя[0]=значение`. Доступ: `${имя[индекс]}`.

4. Каково назначение операторов let и read?

Ответ:
`let` — для арифметических вычислений (например, `let sum=x+7`).
`read` — для чтения ввода пользователя с клавиатуры в переменные.

5. Какие арифметические операции можно применять в языке программирования bash?

Ответ:
`+`, `-`, `*`, `/` (целочисленное деление), `%` (остаток), `**` (возведение в степень), побитовые операции (`&`, `|`, `^`, `<<`, `>>`), сравнения (`<`, `>`, `==`, `!=`), логические (`&&`, `||`, `!`).

6. Что означает операция `(( ))`?

Ответ:
Двойные круглые скобки используются для выполнения арифметических выражений без ключевого слова `let`.
Пример: `(( sum = a + b ))`. Позволяют использовать `++`, `--`, сравнения внутри.

7. Какие стандартные имена переменных Вам известны?

Ответ:
`PATH` — пути поиска команд, `HOME` — домашний каталог, `PS1` — приглашение командной строки, `PS2` — вторичное приглашение, `IFS` — разделители полей, `LOGNAME` — имя пользователя, `TERM` — тип терминала.

8. Что такое метасимволы?

Ответ:
Символы, имеющие специальное значение для оболочки: `*`, `?`, `[ ]`, `;`, `&`, `|`, `<`, `>`, `$`, `\`, `"`, `'`, `` ` ``.
Используются для подстановки, перенаправления ввода/вывода, фонового выполнения, конвейеров.

9. Как экранировать метасимволы?

Ответ:
С помощью `\` перед метасимволом (например, `\*`), или заключением в одинарные кавычки `'...'` (полное экранирование), или в двойные кавычки `"..."` (частичное экранирование, кроме `$`, `` ` ``, `\`, `"`).

10. Как создавать и запускать командные файлы?

Ответ:
Создать текстовый файл с шебангом `#!/bin/bash` в первой строке, сделать исполняемым: `chmod +x файл`, запустить: `./файл`.

11. Как определяются функции в языке программирования bash?

Ответ:

bash

```
function имя {    команды}
```

или

```
bash

имя() {    команды}
```

12. Каким образом можно выяснить, является файл каталогом или обычным файлом?

Ответ:
В условных выражениях: `-d файл` (каталог), `-f файл` (обычный файл).
Пример: `if [ -d "$file" ]; then echo "каталог"; fi`.

13. Каково назначение команд set, typeset и unset?

Ответ:
`set` — выводит все переменные и функции, устанавливает параметры оболочки.
`typeset` — задаёт атрибуты переменных (например, `-i` для целых, `-f` для функций).
`unset` — удаляет переменные или функции (`unset -f имя_функции`).

14. Как передаются параметры в командные файлы?

Ответ:
При вызове скрипта: `./скрипт арг1 арг2 ...`
Внутри скрипта: `$1`, `$2`, … `$9` (позиционные параметры), `$0` — имя скрипта, `$#` — количество аргументов, `$@` — все аргументы.

15. Назовите специальные переменные языка bash и их назначение.

Ответ:
`$0` — имя скрипта,
`$1..$9` — аргументы,
`$#` — количество аргументов,
`$@` и `$*` — все аргументы,
`$?` — код возврата последней команды,
`$$` — PID текущего процесса,
`$!` — PID последней фоновой команды,
`$-` — текущие флаги оболочки.

---

4. Выводы

В ходе выполнения лабораторной работы я:

1.

Изучил основные возможности командной оболочки `bash`.

2.
3.

Научился создавать и запускать исполняемые скрипты.

4.
5.

Освоил работу с переменными, массивами и арифметическими операциями.

6.
7.

Применил управляющие конструкции `for`, `if`, проверку файлов (`-d`, `-f`, `-r`, `-w`, `-x`).

8.
9.

Научился обрабатывать параметры командной строки (включая произвольное количество).

10.
11.

Реализовал аналог команды `ls` без использования самой команды.

12.
13.

Научился использовать `find` и `wc -l` для подсчёта файлов по расширению.

14.
15.

Закрепил навыки архивации с помощью `tar`.

16.

Все 4 задания выполнены полностью, скрипты работают корректно.

Подпись студента: (Аньсяоянь)
Дата: 2026.04.10